Document Management Repository Sprint 2

Lokesh, Rakesh, Ujwal, Ashrith, Bhuvan, Mohit

March 23, 2026

Outline

1. Motivation
2. Problem Statement
3. Technical Explanation
4. Functional Outcomes
5. Working Demonstration
6. Test Scenarios
7. Sprint 2 Summary
8. Sprint 3 Plan

Motivation

Current Challenges in Document Management:

- Documents are scattered across multiple platforms
- Manual organization leads to inefficiency
- Difficulty in searching scanned or unstructured files
- Lack of proper version control and audit tracking
- Limited collaboration and access control mechanisms

Why This Project?

- Need for a centralized and intelligent system
- Growing importance of automation using AI
- Demand for secure and scalable document handling

Problem Statement

Problem Definition:

- Organizations lack a unified platform to efficiently manage documents
- Existing systems fail to provide:
  - Intelligent classification and tagging
  - Secure and flexible access control
  - Proper versioning and audit mechanisms

Proposed Solution:

- Develop a **Document Management Repository (DMR)**
- Features include:
  - Centralized document management system
  - AI-based auto-tagging and classification
  - Role-based access control and sharing
  - Versioning and lifecycle management

System Architecture

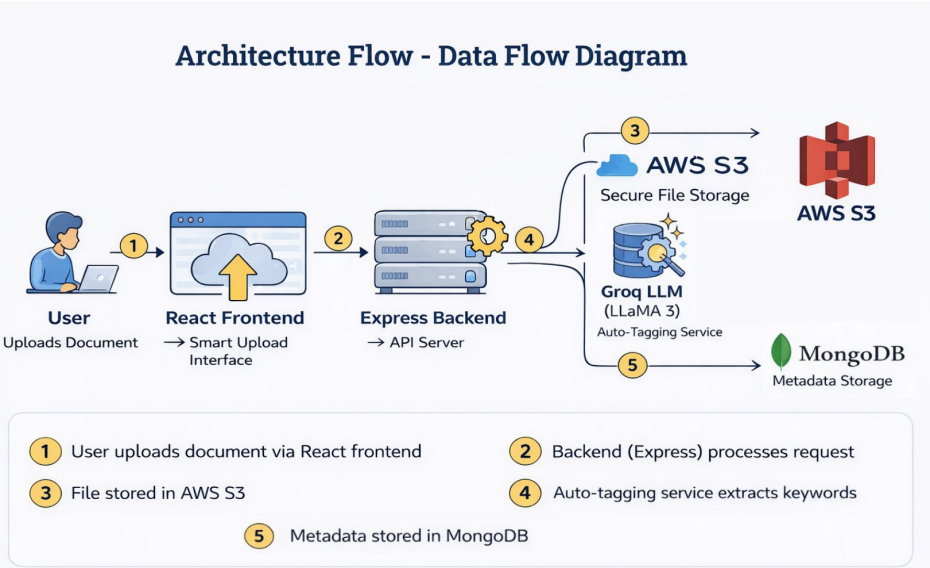
Overall Architecture:

- MERN Stack: MongoDB, Express, React, Node.js
- Cloud Storage: AWS S3 for document storage
- AI Layer: Groq LLM (LLaMA 3) for auto-tagging

Architecture Flow:

- User uploads document via React frontend
- Backend (Express) processes request
- File stored in AWS S3
- Auto-tagging service extracts keywords
- Metadata stored in MongoDB

Data Flow


Architecture Flow - Data Flow Diagram
User
Uploads Document
1
React Frontend
→ Smart Upload Interface
2
Express Backend
→ API Server
4
3
AWS S3
Secure File Storage
AWS S3
Groq LLM
(LLaMA 3)
Auto-Tagging Service
5
MongoDB
Metadata Storage
1 User uploads document via React frontend
2 Backend (Express) processes request
3 File stored in AWS S3
4 Auto-tagging service extracts keywords
5 Metadata stored in MongoDB

Step 1: User Upload (Frontend)

Description:

- User uploads document via Smart Upload Interface
- Drag-and-drop or file selection supported

Technical Details:

- Built using React + Vite
- File wrapped using FormData
- Axios used to send POST request to backend

Output:

- File transmitted to backend API

Step 2: Backend Processing (Express)

Description:

- Backend receives file upload request
- Processes request through API endpoints

Technical Details:

- Node.js + Express server
- Multer middleware handles file parsing
- JWT middleware

Output:

- File buffer ready for storage and processing

Step 3: File Storage (AWS S3)

Description:

- Uploaded file is stored in cloud storage

Technical Details:

- AWS S3 used for scalable object storage
- Unique S3 keys generated for each file
- Secure upload using AWS SDK

Output:

- File stored securely with S3 reference key

Database Schema

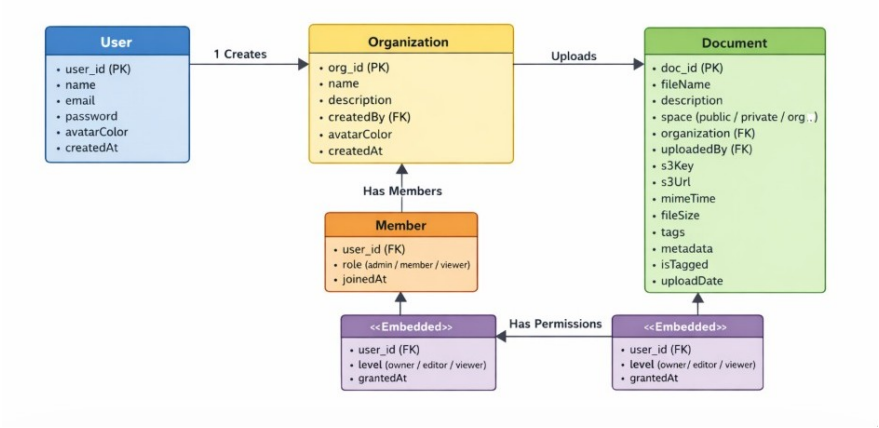


Figure: Schema

Step 4: Auto-Tagging (Groq LLM)

Description:

- Document content analyzed automatically
- Keywords and metadata extracted

Technical Details:

- pdf-parse extracts text from documents
- Groq LLM (LLaMA 3) generates tags
- Structured JSON metadata returned

Output:

- Tags stored and used for classification/search

Step 5: Metadata Storage (MongoDB)

Description:

- Document metadata stored in database

Technical Details:

- MongoDB with Mongoose schemas
- Stores fileName, tags, space, permissions
- Indexed fields for fast search

Output:

- Structured metadata linked to S3 file

Modules & Components

Frontend (React + Vite):

- Smart Upload Interface (Drag-and-drop)
- Dashboard (Document visualization)
- Auth & Theme Context Providers

Backend (Node + Express):

- REST APIs: /auth, /documents, /organizations
- Middleware: JWT Authentication
- Multer for file handling

Core Services:

- Auto-Tagging Service (Groq API + pdf-parse)
- S3 Service (Upload, Download, Delete)
- Storage Quota Manager

Technologies & Tools

Frontend:

- React 18, Vite, Tailwind CSS

Backend:

- Node.js, Express.js
- MongoDB with Mongoose

Other Tools:

- AWS S3 (secure storage)
- Groq API (LLM-based tagging)
- pdf-parse, multer

Technical Challenges & Solutions

1. **Auto-Tagging Accuracy**
   - Challenge: Extracting meaningful metadata
   - Solution: Prompt engineering with Groq LLM
2. **Secure Access Control**
   - Challenge: Multi-user permissions
   - Solution: JWT-based authentication + role-based access
3. **Storage Management**
   - Challenge: Prevent excessive storage usage
   - Solution: Storage quota enforcement logic

Functional Outcomes

Sprint Goal:

- Validate Smart Upload workflow

Completed Features:

- Smart Upload Interface (Drag-and-drop): System accepts raw documents
- Auto-Tagging of documents: Generates tags without manual effort
- Prototype Dashboard (document visualization)

Pending:

- Vault Routing (mapping tags to vaults)

Key Functional Flow:

- Upload $\rightarrow$ Auto-tag $\rightarrow$ Store $\rightarrow$ Display in dashboard

Demonstration

We shall have a live demonstration of our project now.

Test Scenarios

1. **Upload Test**
   - Input: PDF document
   - Expected: Successful upload + stored in S3
2. **Auto-Tagging Test**
   - Input: Document with keywords (e.g., invoice)
   - Expected: Tags generated correctly
3. **Dashboard Display**
   - Expected: Document appears in UI with tags
4. **Access Control**
   - Expected: Only authorized users can access private docs

Results Summary

Observations:

- Upload and tagging pipeline works end-to-end
- UI successfully reflects categorized documents
- AI tagging improves document discoverability

Limitations:

- Vault routing has not been implemented yet

Conclusion:

- Core workflow has been validated successfully

Sprint 2: Goal & Deliverables

The objective of Sprint 2 is to validate the **Smart Upload Workflow**. The system should automatically:

- Accept raw documents from users
- Analyze document content
- Auto-tag files based on keywords
- Store documents in the correct vault (Finance / HR / Project)

Deliverables Completed:

- **Smart Upload Interface:** React-based drag-and-drop upload system
- **Auto-Tagging Service:** Node.js service using `pdf-parse`
- **Vault Router:** Maps extracted tags to MongoDB collections
- **Prototype Dashboard:** Displays categorized uploaded documents

Key Outcome: Fully functional end-to-end automated document pipeline eliminating manual sorting.

Sprint 2: Team Contributions

Ujwal (Team Lead & Architect)

- Led frontend-backend integration
- Managed GitHub workflows and code reviews

Madhava (Backend Developer – Core Logic)

- Built Node.js/Express server
- Implemented file uploads using Multer
- Developed auto-tagging using `pdf-parse`

Bhuvan (Backend Developer – Auth & Security)

- Implemented JWT-based authentication
- Developed login/signup APIs
- Secured routes using middleware

Sprint 2: Team Contributions

Rakesh (Frontend Developer)

- Setup React with Tailwind CSS
- Built login and upload UI components
- Integrated frontend with backend APIs
- Displayed auto-detected tags after upload

Lokesh (Database Engineer)

- Setup MongoDB (Atlas/Local)
- Designed User and Document schemas
- Integrated secure uploading and retrieval using AWS S3

Mohit (QA & Documentation)

- Tested authentication and token flow
- Validated auto-tagging accuracy
- Verified correct document storage

Sprint 3: Plan

Sprint Focus Areas:

- **Bulk Uploads:** Enable users to upload multiple documents efficiently in a single operation.
- **Deployments:** Deploy the full application (frontend + backend) to a production-ready environment.
- **Vault Router:** Enhance routing logic to correctly classify and store documents based on tags.
- **Improvements in UI:** Refine user interface for better usability, responsiveness, and feedback.
- **Completion of Organizational Space(In Progress):** Finalize structure for organizing documents across users, roles, and vaults.